

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengenalan Mikroprosesor

La Rangga Abidzar Al Ghifari - 5024241064

2026

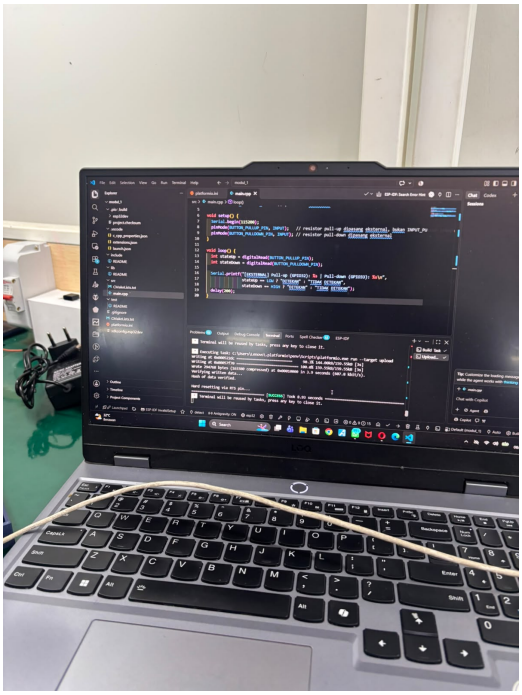
1 Langkah-Langkah Percobaan

1.1 Percobaan 1: Blink pada STM32 Blackpill

Tujuan: Mengenali board STM32 Blackpill dan mengunggah program Blink menggunakan PlatformIO, framework Arduino, dan ST-Link.

Langkah kerja:

1. Buat atau buka projek Modul 1 di VS Code. Atur `platformio.ini` sesuai konfigurasi di atas dan tunggu proses refresh/re-index PlatformIO selesai.
2. Hubungkan ST-Link ke Blackpill melalui SWDIO, SWCLK, GND, dan 3.3V. Pastikan tegangan target 3.3V dan sambungan pin sesuai label pada kedua perangkat.
3. Tulis program Blink berikut pada main.cpp, kemudian jalankan Build dan Upload.
4. Amati LED.



Gambar 1: Build dan Upload berstatus berhasil; kode pada layar membaca tombol di GPIO 32 dan 33.

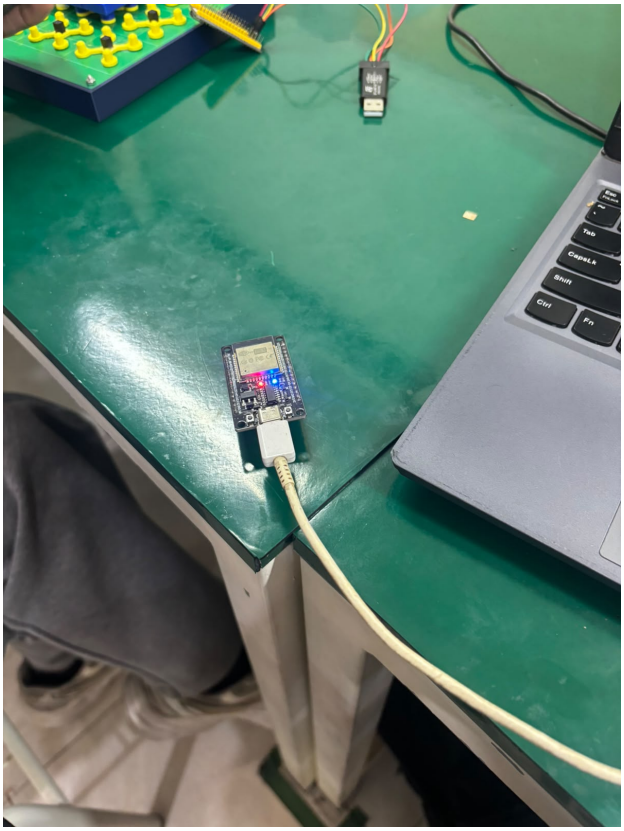
1.2 Percobaan 2: Blink ESP32 dengan ESP-IDF

Tujuan: Mengenali struktur program ESP-IDF dan menjalankan Blink pada ESP32 melalui PlatformIO.

Langkah kerja:

1. Ganti konfigurasi proyek menjadi konfigurasi ESP32 di atas dan tunggu hingga PlatformIO selesai memperbarui konfigurasi.
2. Hubungkan board ESP32 ke komputer menggunakan kabel USB dan pastikan port serialnya terdeteksi.
3. Hapus main.cpp bila ada, buat `src/main.c`, lalu tulis program di bawah.

4. Jalankan Build dan Upload. Setelah proses selesai, amati LED pada GPIO 2; program mengubah level LED setiap 500 ms.



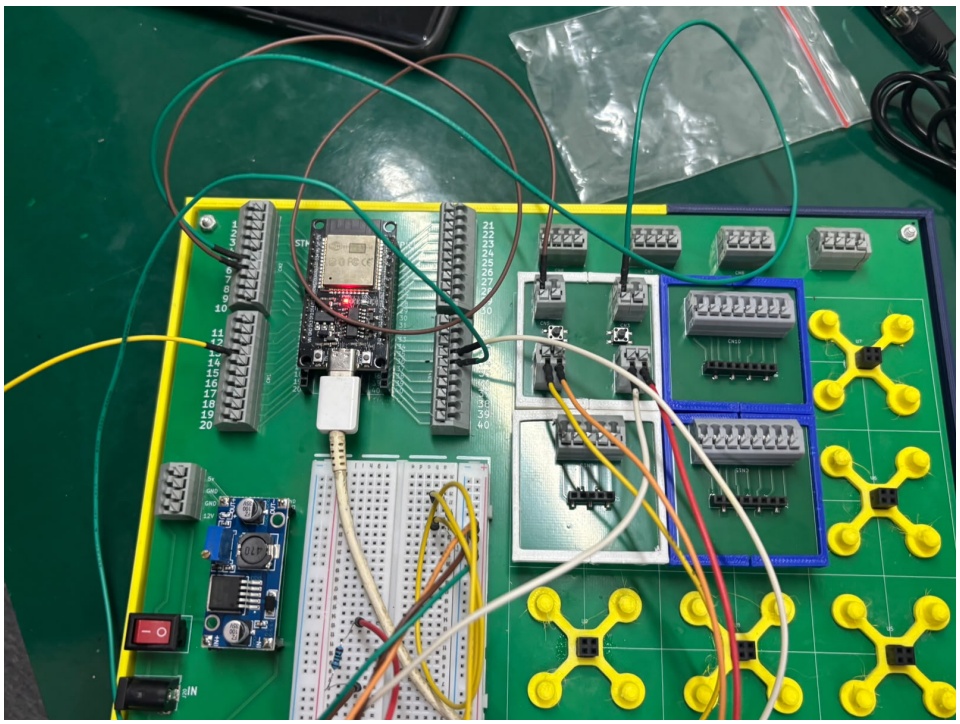
Gambar 2: Board ESP32 tersambung ke komputer melalui USB untuk percobaan ESP-IDF.

1.3 Percobaan 3: Pull-Up dan Pull-Down Eksternal serta Internal

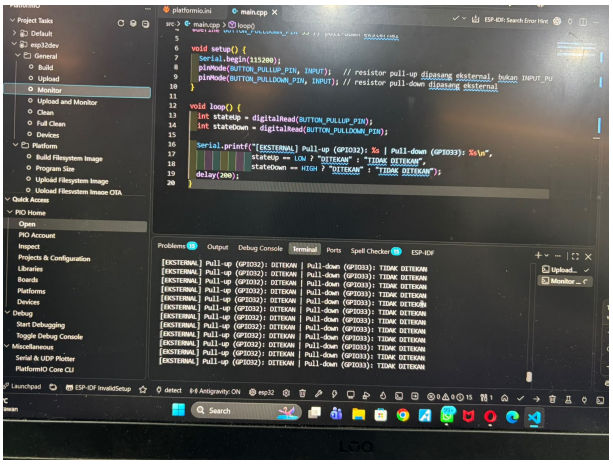
Tujuan: Membandingkan dua tombol pada GPIO 32 dan GPIO 33 dengan resistor pull-up/pull-down eksternal dan konfigurasi pull-up/pull-down internal ESP32.

Langkah kerja:

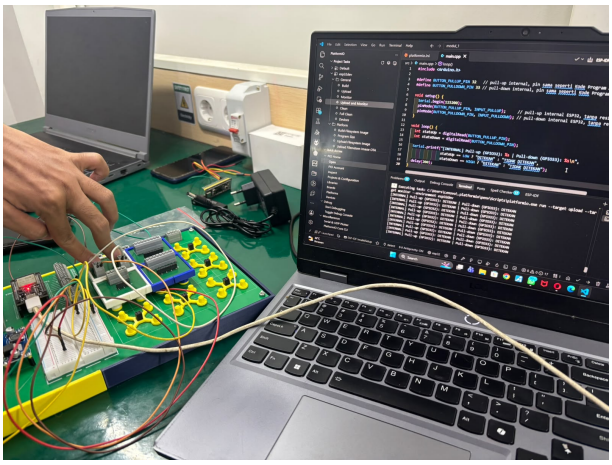
1. Atur proyek ke konfigurasi Arduino di atas. Tunggu refresh PlatformIO selesai sebelum Build atau Upload.
2. upload Kode A. Tekan dan lepaskan kedua tombol secara bergantian, lalu amati statusnya pada Serial Monitor dengan baud rate 115200.
3. Untuk pengujian internal, lepaskan resistor eksternal, pertahankan kedua tombol pada pin yang sama, dan gunakan rangkaian pull-up/pull-down internal. Pastikan orientasi kaki pushbutton benar.
4. upload Kode B dan ulangi pengamatan. Bandingkan keadaan default dan keadaan saat tombol ditekan pada kedua konfigurasi.



Gambar 3: Rangkaian tombol dan board pada pengujian pull-up/pull-down.



(a) Keluaran Serial Monitor pada pengujian eksternal.



(b) Keluaran Serial Monitor pada pengujian internal.

Gambar 4: Perbandingan pembacaan tombol dengan konfigurasi pull eksternal dan internal.

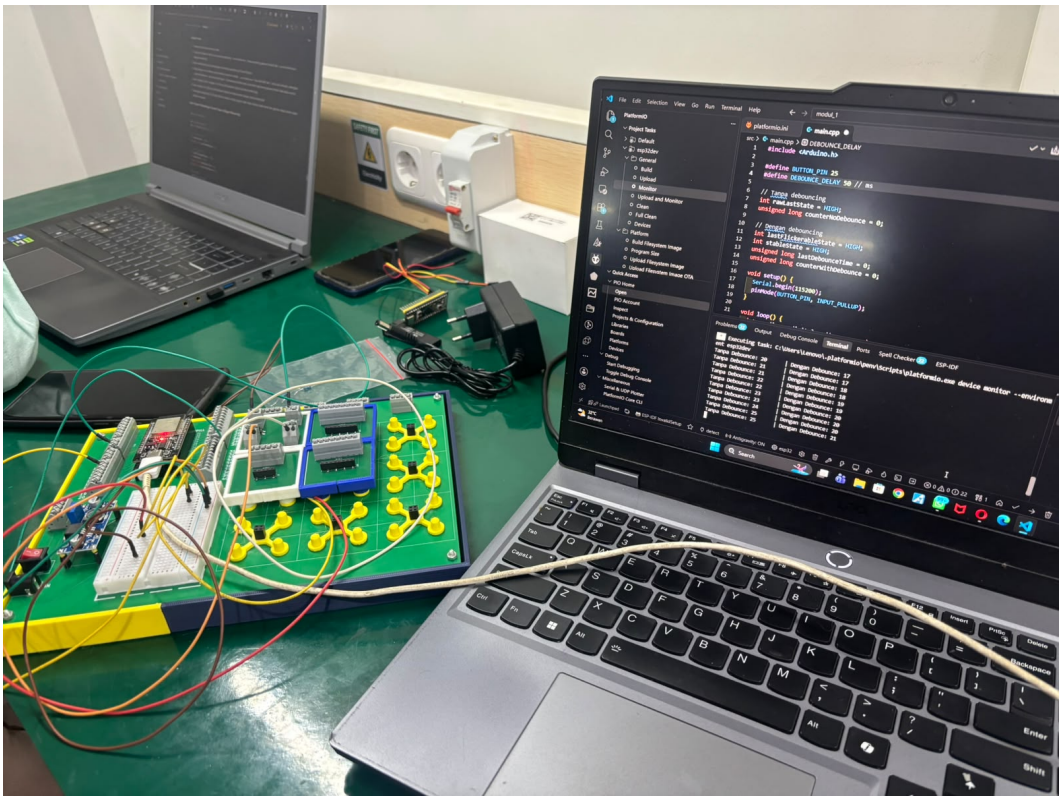
1.4 Percobaan 4: Debouncing Tombol GPIO

Tujuan: Mengamati bouncing pada tombol mekanik dan membandingkan pencacahan tanpa debouncing dengan debouncing perangkat lunak.

Langkah kerja:

1. Hubungkan pushbutton antara GPIO 25 dan GND, kemudian atur pin sebagai `INPUT_PULLUP`.
2. Unggah program berikut. Tekan tombol satu kali dan bandingkan kedua counter pada Serial Monitor.
3. Ulangi penekanan dengan kecepatan berbeda. Ubah `DEBOUNCE_DELAY` menjadi 10 ms, 50 ms, dan 200 ms; setiap perubahan perlu di-build dan diunggah kembali.

4. Bandingkan jumlah hitungan serta respons tombol untuk setiap nilai waktu debouncing.



Gambar 5: Program debouncing dan perbandingan counter pada Serial Monitor.

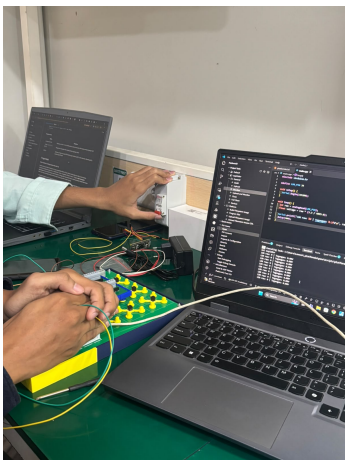
1.5 Percobaan 5: Pembacaan ADC dengan LDR

Tujuan: Membaca perubahan intensitas cahaya menggunakan LDR yang dihubungkan sebagai pembagi tegangan ke ADC ESP32.

Langkah kerja:

1. Hubungkan LDR ke 3.3V. Sambungkan titik tengah LDR dan resistor 10 ohm ke GPIO 34, lalu hubungkan ujung resistor lainnya ke GND.

2. Unggah program berikut ke ESP32 dengan framework Arduino dan buka Serial Monitor pada baud rate 115200.



(a) Pengamatan dengan sensor pada kondisi minim cahaya.



(b) Pengamatan saat LDR disinari lampu ponsel.

Gambar 6: Pembacaan ADC LDR pada kondisi pencahayaan berbeda.

2 Analisis Hasil Percobaan

2.1 Percobaan 1: Blink STM32 Blackpill

Dokumentasi terbaru menunjukkan status Build dan Upload `SUCCESS`, sehingga proses kompilasi dan pengiriman firmware pada proyek yang tampak telah berhasil. Namun, kode pada layar menggunakan GPIO 32 dan GPIO 33 untuk membaca tombol, bukan program Blink STM32 pada PC13. Karena itu, foto ini belum membuktikan program Blink berjalan pada Blackpill ataupun LED berkedip. Keberhasilan Blink PC13 perlu dikonfirmasi dengan mengunggah program Blink ke Blackpill dan mengamati LED onboard.

2.2 Percobaan 2: Blink ESP32 dengan ESP-IDF

Program ESP-IDF menginisialisasi GPIO 2 sebagai keluaran dan memakai `vTaskDelay` untuk memberi jeda 500 ms di antara perubahan level. Foto menunjukkan board ESP32 terhubung ke komputer melalui USB, tetapi tidak menampilkan log Build/Upload atau keluaran LED yang dapat memastikan program berjalan. Secara teori LED pada GPIO 2 akan berubah keadaan tiap setengah detik; keberhasilan pelaksanaan perlu dipastikan melalui status Upload dan pengamatan LED pada board yang digunakan.

2.3 Percobaan 3: Pull-Up dan Pull-Down

Keluaran pada foto pengujian eksternal menunjukkan GPIO 32 terbaca `DITEKAN` dan GPIO 33 `TIDAK DITEKAN`. Pada foto konfigurasi internal, kedua masukan ditampilkan sebagai `DITEKAN`. Ini menunjukkan program berhasil membaca perubahan logika dari rangkaian tombol pada kondisi yang terekam. Pull-up seharusnya bernilai HIGH saat tidak ditekan dan LOW saat ditekan, sedangkan pull-down bernilai LOW saat tidak ditekan dan HIGH saat ditekan. Foto hanya merekam kondisi tombol tertentu, sehingga pengujian tambahan dengan kedua tombol dilepas diperlukan untuk membandingkan seluruh keadaan default dan memastikan hasil eksternal serta internal konsisten.

2.4 Percobaan 4: Debouncing

Program membandingkan hitungan transisi tombol mentah dengan hitungan yang hanya diterima setelah sinyal stabil selama `DEBOUNCE_DELAY`. Pada foto Serial Monitor, hitungan tanpa debouncing mencapai 25, sedangkan hitungan dengan debouncing mencapai 21. Selisih tersebut memperlihatkan bahwa beberapa transisi singkat pada kontak tombol dapat terhitung sebagai penekanan tambahan tanpa penyaringan. Nilai 50 ms yang terlihat pada kode menghasilkan keluaran yang lebih stabil pada pengamatan ini. Pengujian 10 ms dan 200 ms perlu dibandingkan terpisah: waktu terlalu pendek dapat membiarkan bouncing lolos, sedangkan waktu terlalu panjang dapat memperlambat respons dan mengabaikan penekanan cepat.

2.5 Percobaan 5: ADC pada LDR

Serial Monitor memperlihatkan pembacaan ADC mentah dan estimasi tegangan. Pada foto saat minim cahaya, nilai yang terlihat mendekati 0 dan tegangan 0.00 V; saat LDR disinari lampu ponsel, pembacaan meningkat mendekati skala penuh dan sekitar 3.3 V. Perubahan ini sesuai dengan rangkaian pembagi tegangan yang digunakan: intensitas cahaya mengubah resistansi LDR, sehingga tegangan pada titik tengah ikut berubah dan dikonversi ADC menjadi kode digital. Nilai tersebut merupakan hasil rangkaian dan kalibrasi yang digunakan, sehingga pembacaan tidak langsung menjadi ukuran intensitas cahaya absolut.

3 Hasil Tugas Modul

Program Matrix Pushbutton dan LED

Program berikut melakukan pemindaian matriks pushbutton 3×3. Baris diaktifkan bergantian dengan logika LOW, sedangkan kolom dibaca menggunakan masukan pull-up. Status tombol disaring menggunakan debouncing perangkat lunak selama 50 ms. Setiap LED mengikuti status tombol pada posisi yang bersesuaian.

Listing 1: Program pemindaian matrix pushbutton dan LED

```
const int rowPins[3] = {13, 14, 27};

const int colPins[3] = {25, 26, 33};

const int ledPins[3][3] = {
  {16, 17, 18},
  {19, 21, 22},
  {23, 2, 4}
};

bool stableState[3][3] = {false};

bool lastReading[3][3] = {false};

unsigned long lastDebounceTime[3][3] = {0};

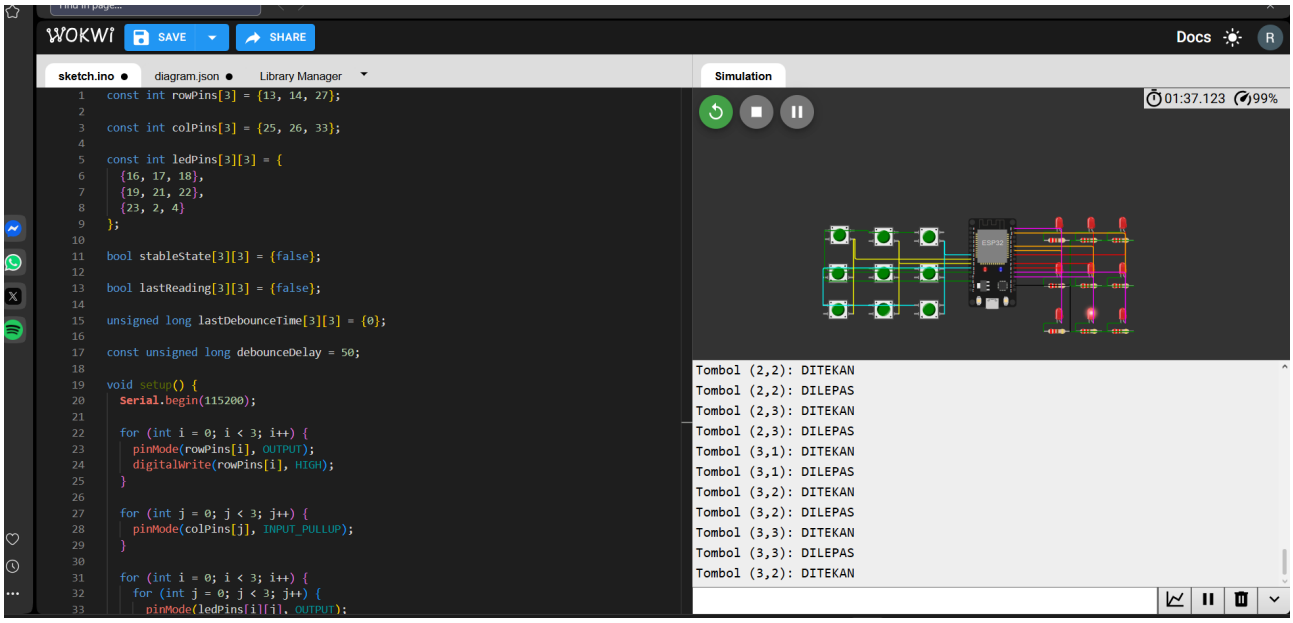
const unsigned long debounceDelay = 50;

```

```
19 void setup() {
20   Serial.begin(115200);
21
22   for (int i = 0; i < 3; i++) {
23     pinMode(rowPins[i], OUTPUT);
24     digitalWrite(rowPins[i], HIGH);
25   }
26
27   for (int j = 0; j < 3; j++) {
28     pinMode(colPins[j], INPUT_PULLUP);
29   }
30
31   for (int i = 0; i < 3; i++) {
32     for (int j = 0; j < 3; j++) {
33       pinMode(ledPins[i][j], OUTPUT);
34       digitalWrite(ledPins[i][j], LOW);
35     }
36   }
37 }
38
39 void loop() {
40
41   for (int i = 0; i < 3; i++) {
42
43     digitalWrite(rowPins[i], LOW);
44
45     delayMicroseconds(10);
46
47     for (int j = 0; j < 3; j++) {
48
49       bool reading = (digitalRead(colPins[j]) == LOW);
50
51       if (reading != lastReading[i][j]) {
52         lastDebounceTime[i][j] = millis();
53         lastReading[i][j] = reading;
54       }
55
56       if (millis() - lastDebounceTime[i][j] >= debounceDelay) {
57
58         if (reading != stableState[i][j]) {
59           stableState[i][j] = reading;
60
61           Serial.print("Tombol (");
62           Serial.print(i + 1);
63           Serial.print(",");
64           Serial.print(j + 1);
65           Serial.print("): ");
66
67           Serial.println(reading ? "DITEKAN" : "DILEPAS");
68         }
69       }
70
71       digitalWrite(
72         ledPins[i][j],
```

```
        stableState[i][j] ? HIGH : LOW
      );
    }

    digitalWrite(rowPins[i], HIGH);
  }
}
```



Gambar 7: Simulasi rangkaian matrix pushbutton, ESP32, LED, dan keluaran serial.

1. Analisis mengapa debouncing tetap diperlukan pada rangkaian matrix pushbutton ini, walaupun pada satu waktu hanya satu tombol yang biasanya ditekan (kaitannya dengan C.5 Debouncing).

   Jawaban: Debouncing tetap diperlukan karena bouncing berasal dari kontak mekanis di dalam setiap pushbutton, bukan karena jumlah tombol yang ditekan bersamaan. Saat tombol ditekan atau dilepas, kontak dapat memantul beberapa kali sebelum stabil, sehingga sinyal berubah-ubah antara HIGH dan LOW selama beberapa milidetik. Rentang pastinya bergantung pada jenis dan kondisi tombol; pada program ini waktu stabil yang digunakan adalah 50 ms melalui variabel `debounceDelay`.

2. Analisis konfigurasi pull-up atau pull-down yang digunakan pada pembacaan kolom tombol, kondisi (HIGH/LOW) saat tombol tidak ditekan, dan alasan tidak boleh dibiarkan floating.

   Jawaban: Pin kolom `colPins = {25, 26, 33}` dikonfigurasi sebagai `INPUT_PULLUP`. Karena itu, saat tombol tidak ditekan, resistor pull-up internal menjaga pin kolom pada HIGH. Ketika suatu baris sedang dipindai dengan logika LOW dan tombol pada persilangan baris-kolom tersebut ditekan, pin kolom terhubung ke baris LOW dan terbaca LOW. Konfigurasi ini disebut active-low.

   Jika pin kolom dibiarkan tanpa pull-up atau pull-down, pin dapat berada dalam keadaan floating karena tidak memiliki level tegangan acuan yang pasti. Gangguan listrik atau kopling dari jalur baris yang berubah saat pemindaian dapat mengubah pembacaan secara tidak menentu. Pembacaan palsu tersebut dapat menyebabkan status tombol dan LED berubah meskipun tombol tidak ditekan. Resistor pull-up menetapkan keadaan default HIGH agar masukan digital lebih stabil.

3. Analisis perbedaan mendasar antara membaca GPIO sebagai Digital Input (tombol, hanya HIGH/LOW) dan sebagai Analog Input (ADC pada LDR), serta alasan ADC dapat merepresentasikan lebih dari dua kondisi.

   Jawaban: GPIO digital dibaca sebagai salah satu dari dua keadaan, HIGH atau LOW. Tegangan masukan dibandingkan dengan ambang logika digital. Pada ESP32 dengan catu I/O 3,3 V, batas yang umum dinyatakan pada datasheet adalah VIL maksimum sekitar 0,25 VDD (sekitar 0,825 V) untuk LOW dan VIH minimum sekitar 0,75 VDD (sekitar 2,475 V) untuk HIGH. Tegangan di antara batas tersebut tidak dijamin terbaca sebagai salah satu keadaan tertentu. Pembacaan digital sesuai untuk tombol yang merepresentasikan keadaan tekan atau lepas.

4 Kesimpulan

Praktikum ini memberikan pemahaman dasar pemrograman STM32 dan ESP32, konfigurasi pull-up/pull-down, debouncing tombol, serta pembacaan ADC pada LDR. Pengujian menunjukkan bahwa kestabilan masukan dan konfigurasi board yang tepat penting agar program dapat berjalan dan menghasilkan pembacaan yang sesuai.

5 Lampiran

5.1 Dokumentasi saat praktikum

